Contents

1 Introduction 1
1.1 Introduction 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Scope and Limitation 2
1.5 Development Methodology 4
1.6 Report Organization 4
2 Background Study and Literature Review 6
2.1 Background Study 6
2.2. Literature Review 7
3 System Analysis 9
3.1 System Analysis 9
3.1.1 Requirement Analysis 9
3.1.2 Feasibility Analysis 12
3.1.3 Analysis 13
4 System Design 17
4.1 Design 17
4.1.1 Database Design: Transformation of ER to Relations and Normalizations 18
FORM AND REPORT DESIGN 23
..... 23
4.2 Algorithm Details 25
5 Implementation 26
5.1 Implementaion 26
5.1.1 Tools Used 26
5.1.2 Implementation Details of Modules 26
5.2 Testing 28
5.2.1 Test Cases for Unit Testing 28
5.3 Result Analysis 30
6 References 31

1 Introduction

1.1 Introduction

Educational institutes rely primarily on manual methods for recording attendance and maintaining student records. The College Management System is a web application integrated with Machine Learning to manage student records and predict final grades. By combining digital record-keeping with predictive analytics, this system transforms manual monitoring into a centralized web-based platform.

The system utilizes a multi-actor architecture involving Administrators, Teachers, and Students. Incorporating functionalities such as assignment distribution, student uploads, and fee payment tracking streamlines the college workflow, replacing manual processes with a digital framework. Furthermore, role-specific dashboards consolidate data and insights for direct observation. Teachers can access student record summaries and academic trends, while administrators can monitor college metrics and system activities from a centralized interface.

To support academic monitoring, the system employs Linear Regression and Random Forest models to predict final grades. The Random Forest model identifies the influential factors affecting student performance and provides data-driven insights for academic improvement. Predicting outcomes in advance enables the early identification of at-risk students, allowing for intervention prior to final assessments.

1.2 Problem Statement

Educational institutes rely primarily on manual methods for attendance tracking and student record maintenance. Existing digital tools often fragment academic workflows, such as assignment distribution and submission, and administrative tasks, such as fee processing. This lack of integration creates operational inefficiencies and leaves administrators and teachers without centralized dashboards to monitor institutional metrics or view student records systematically.

Furthermore, current systems typically function as static data repositories without incorporating predictive analytics. Consequently, students at risk of underperforming are not identified until after final assessments are completed. There is a structural need for a

centralized platform that unifies academic and administrative modules while utilizing Machine Learning models to predict student performance prior to final examinations. This predictive capability is necessary to provide early warnings and data-driven insights, enabling timely academic intervention before final grades are determined.

1.3 Objectives

The main objectives of this project are:

- To develop a web-based system integrating student record management, assignment tracking, and fee processing to streamline overall administrative and academic operations.
- To forecast student final grades using machine learning models and identifying key performance factors to provide early warnings and actionable insights prior to final exams.
-

1.4 Scope and Limitation

Scope:

- **Role-Based Dashboards:** The system provides Administrators, Teachers, and Students with secure, role-based access. Each actor utilizes a specific dashboard that visualizes relevant data, summaries, and system insights.
- **Management Modules:** Includes student profile management, attendance tracking, an assignment module for distribution and submission, and a fee payment tracking system.
- **Predictive Analytics:** Implements Random Forest and Linear Regression models to forecast final grades based on periodic assessments, attendance, and assignments. The system extracts feature importance to identify the specific variables affecting a student's academic performance.

Limitations:

- **Data Dependency:** The accuracy of the machine learning grade predictions depends directly on the volume and quality of historical and current data entered into the system.

- **Financial Transactions:** The fee payment module will utilize simulated transactions or sandbox payment gateways rather than live banking integrations.
- **Prediction Application:** The machine learning insights serve an advisory purpose; they identify areas for improvement based on data but do not guarantee final academic outcomes.

1.5 Development Methodology

The Agile development model will be used as the software development methodology for the College Management System. Its iterative nature allows different modules and features to be added incrementally. In the initial sprint, core student record management will be implemented. Subsequent sprints will focus on integrating the new student interactions (assignment uploads and fee payments), building the teacher and admin dashboards, and incorporating the Machine Learning components. Finally, in the last sprints, overall integration will be completed and the User Interface will be refined along with hosting.

1.6 Report Organization

Chapter 1: Introduction This initial chapter sets the foundation for the document by providing an overview of the project. It outlines the background, formally defines the problem statement, and establishes the primary objectives. Furthermore, it delineates the project's scope and limitations, and introduces the development methodology adopted to guide the software engineering process.

Chapter 2: Background Study and Literature Review This chapter explores the theoretical framework and prior research relevant to the project. It details fundamental theories and terminologies associated with machine learning and digital academic management. Additionally, it examines similar existing systems and relevant literature to establish the context and necessity for the proposed solution.

Chapter 3: System Analysis This chapter provides a preliminary analysis of the proposed system based on a structured methodology. It begins with requirement analysis, illustrating functional requirements through a use case diagram, and includes a feasibility study evaluating technical, operational, economic, and schedule constraints. The structural system modeling is then represented using Data Flow Diagrams (DFD Level 0 and DFD Level 1) for process modeling and Entity-Relationship (ER) diagrams for data modeling.

Chapter 4: System Design Building upon the structured analysis, this chapter translates system requirements into a technical blueprint. It details the database design through the relational database schema and outlines the system architecture, which is divided into three

core modules. Additionally, this chapter provides the algorithmic details, focusing specifically on the logic of the Random Forest machine learning model used for grade prediction.

Chapter 5: Implementation and Testing This chapter discusses the practical execution of the project. It lists the development tools, programming languages, and databases utilized, followed by the implementation details of the three system modules. The testing section is dedicated exclusively to unit testing to verify the functionality of individual components, accompanied by a result analysis.

Chapter 6: Conclusion and Future Recommendations The final chapter concludes the report by summarizing the project outcomes and evaluating the fulfillment of the initial objectives. It provides a final assessment of the system's effectiveness and offers strategic recommendations for future enhancements

2 Background Study and Literature Review

2.1 Background Study

This section details the fundamental theories, general concepts, and terminologies utilized in the development of the College Management System with Machine Learning-based grade prediction.

College Management Systems (CMS) A College Management System is a centralized, web-based software application designed to digitize and manage the administrative, academic, and financial operations of an educational institution. Core components typically include student profile management, attendance tracking, assignment distribution, and fee processing. By utilizing a multi-actor architecture, a CMS provides role-specific interfaces (dashboards) for administrators, teachers, and students, thereby standardizing workflows and replacing fragmented manual record-keeping.

Educational Data Mining and Predictive Analytics Educational Data Mining (EDM) involves applying mathematical models to data generated by educational settings to understand and improve student learning outcomes. Predictive analytics in this context utilizes historical student data—such as attendance rates, periodic assessment scores, and assignment completion—to forecast future performance, specifically the final grade. This allows institutions to implement early warning systems for academically at-risk students.

Supervised Machine Learning The predictive component of this system relies on Supervised Machine Learning. In this paradigm, an algorithm is trained on a labeled dataset, meaning the input data (features like attendance and assignment scores) is paired with the correct historical output (final grades). The model learns the mapping function from the inputs to the outputs to predict outcomes for unseen, future data.

Random Forest (RF) Algorithm Random Forest is a robust ensemble learning method utilized for regression and classification tasks. It operates by constructing a multitude of decision trees during the training phase. For regression tasks, the output is the average prediction of the individual trees. This method utilizes "bagging" (bootstrap aggregating) to reduce overfitting and improve variance. A critical theoretical component of Random Forest for this project is its ability to calculate Feature Importance. The algorithm mathematically measures how much each feature decreases the model's error, thereby identifying the specific factors most influential to a student's academic performance.

Dataset Specification To train and evaluate the Random Forest model, the system utilizes the "Student Performance Prediction" dataset sourced from Kaggle. This open-source dataset contains structured records encompassing student demographics, study habits, absenteeism, and periodic academic assessments mapped to final academic outcomes. Utilizing this standardized dataset allows the machine learning model to effectively learn the complex, non-linear relationships between behavioral or academic inputs and the target final grade. This ensures the predictive algorithm is robust and well-calibrated before being applied to the real-time data generated within the proposed College Management System.

2.2. Literature Review

The integration of Machine Learning (ML) into educational systems has significantly enhanced the capability to predict student academic performance and identify at-risk learners at early stages.

Ensemble learning techniques, particularly Random Forest (RF), have demonstrated superior performance in academic prediction tasks. Agyemang et al. [1] reported that RF achieved higher predictive accuracy compared to alternative machine learning approaches due to its ability to capture nonlinear relationships and reduce overfitting through bootstrap aggregation. Similarly, Xu and Yang [4] concluded that RF outperforms traditional baseline models when interdependencies exist among predictive factors.

Beyond predictive algorithms, the foundational architecture of the management platform plays a critical role in educational administration. Literature concerning traditional web-based College Management Systems [5] highlights the operational efficiency gained by integrating academic and administrative modules. Studies on these architectures illustrate that centralizing student records, assignment workflows, and fee processing significantly reduces administrative overhead and improves information accessibility for both students and faculty. However, while these conventional platforms excel at data organization, they traditionally serve as static repositories and lack the predictive analytical capabilities required to proactively support student learning.

Bridging this gap, research in *IEEE Transactions on Learning Technologies* [3] highlights the use of predictive analytics to identify students at risk of dropout through early academic signals. AI-driven student success predictors discussed in *Computers in Human Behavior*

[2] demonstrate high forecasting accuracy when sufficient and well-structured data is available, facilitating accurate systems and reliable academic interventions.

In the local context, existing Nepali platforms such as Paathshala [6] provide essential College Management System functionalities, including student profile management and attendance tracking. However, they do not integrate machine learning-based grade prediction or early warning analytics for identifying academically at-risk students. In many regional colleges, digital systems function merely as static data repositories without analytical capabilities. This creates a functional gap between data collection and the extraction of actionable academic insights, which the proposed system aims to resolve.

3 System Analysis

3.1 System Analysis

3.1.1 Requirement Analysis

i. Functional Requirements Functional requirements define the specific behaviors, operations, and information processing that the system must perform. The functional requirements of this system involve three primary actors: Administrators, Teachers, and Students.

- **User Registration & Authentication:** Secure account creation, role-based access control, and login mechanisms for Administrators, Teachers, and Students.
- **Administrator Module (Dashboard & Management):** * Register new students and manage comprehensive student profiles.
    - Manage teacher accounts and system access.
    - Access an overarching admin dashboard to view system-wide insights and data summaries at a glance.
- **Teacher Module (Academic Management):**
    - Input and update student attendance and periodic assessment marks (G1, G2).
    - Distribute assignments to respective classes or students.
    - Access a dedicated teacher dashboard to view individual student record summaries and track academic trends.
- **Student Module (Engagement & Finance):**
    - View distributed assignments and upload completed assignment files.
    - Process and track academic fee payments.
    - View personal academic progress and attendance records.
- **Machine Learning Grade Prediction (G3):**
    - Predict the final grade (G3) of students based on historical and current data (attendance, G1, G2, assignment completion) utilizing the Random Forest algorithm.

- **Performance Factor Analysis:**
    - Utilize the Feature Importance function of the Random Forest model to identify the most influential factors affecting each individual student's performance.
    - Generate actionable insights, advising students on which specific areas (e.g., attendance, specific assignments) to improve to prevent grade depletion.

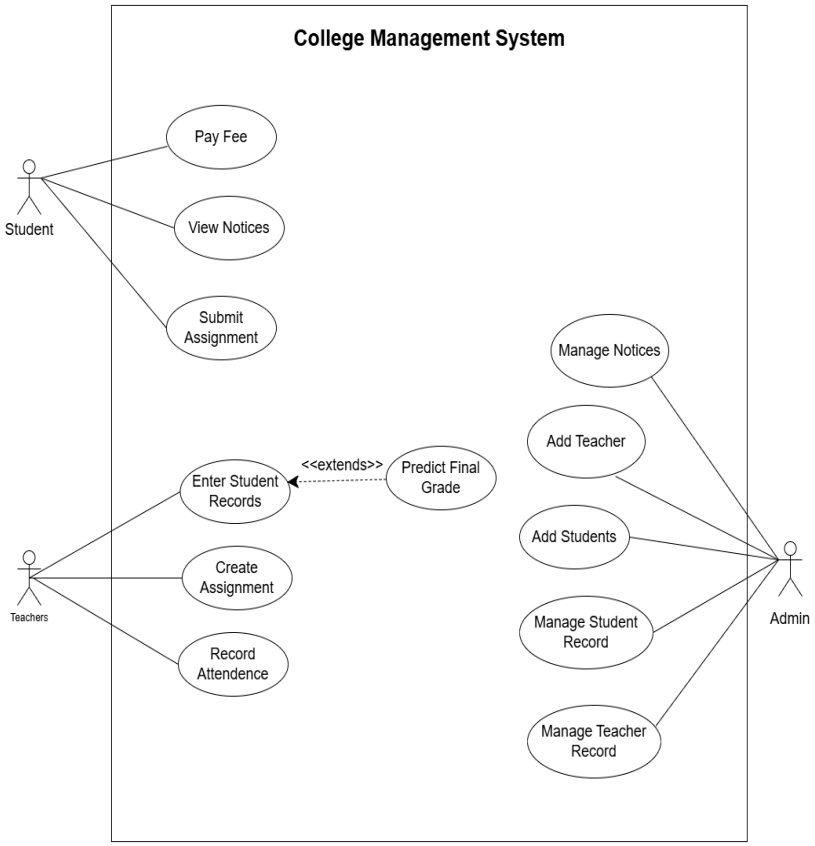


Figure 1 Use Case Diagram of College Management System

ii. Non-Functional Requirements Non-functional requirements specify the quality attributes, performance goals, and security parameters of the system.

- **Reliability and Data Integrity:** The system must utilize a safe, robust database structure to ensure academic and financial data is stored reliably without loss or corruption.
- **Performance and Speed:** The web application should load pages quickly, and the machine learning grade predictions must be generated and displayed in under 30 seconds. Furthermore, the system must smoothly handle concurrent sessions (e.g., 50+ students and multiple teachers accessing the system simultaneously).
- **Security:** Passwords must be securely hashed, and strict role-based authentication must be enforced so that users can only access data authorized for their specific role.
- **Availability:** The web system should maintain a 99% uptime during standard academic semesters to ensure constant accessibility for assignments and administrative tasks.
- **Usability:** The interface must be highly intuitive, ensuring that users with minimal technical expertise can easily navigate dashboards, upload assignments, or input grades.

3.1.2 Feasibility Analysis

i. Technical Feasibility The project relies on well-established, open-source technologies with extensive documentation. React.js will handle the responsive frontend web interface, while Node.js with Express manages the backend logic and API routes. SQLite/PostgreSQL will be utilized to ensure reliable data storage. The predictive modeling will be executed using the Scikit-learn library in Python to deploy the Random Forest algorithm. Given the availability of these tools and the team's familiarity with them, the project is highly technically feasible.

ii. Operational Feasibility The proposed web-based system is designed to seamlessly integrate into daily college operations. Because it features a user-friendly interface, it does not require advanced technical training for staff or students. Teachers can input records effortlessly, students can manage assignments and fees intuitively, and administrators can

oversee operations through centralized dashboards. This ensures smooth operational adoption within a campus environment.

iii. Economic Feasibility The development of this system is highly economically feasible. There is no requirement for specialized external hardware, as the platform is accessible via standard student and staff devices (PCs, laptops, smartphones). Furthermore, the core development technologies (React, Node.js, Python) are open-source and free to use. Model training utilizes the open-source Kaggle dataset, and initial deployment will leverage free or low-cost academic hosting tiers, keeping the budget strictly within feasible limits.

iv. Schedule Feasibility The project utilizes an Agile development methodology, breaking the system down into manageable sprints. By prioritizing core modules first (student management and authentication), followed by secondary modules (assignments, fees, dashboards) and finally the machine learning integration, the workload is evenly distributed. This structured, time-bound approach ensures that the entire project can be completed, tested, and deployed within the designated timeframe of the academic semester.

3.1.3 Analysis

To comprehensively analyze the system's operational mechanisms, a structured methodology was adopted. This involves process modeling to map the flow of information and data modeling to structure the underlying database.

Process Modelling using DFD (Data Flow Diagram) Data Flow Diagrams are utilized to represent the flow of information within the College Management System.

DFD Level 0 (Context Diagram):

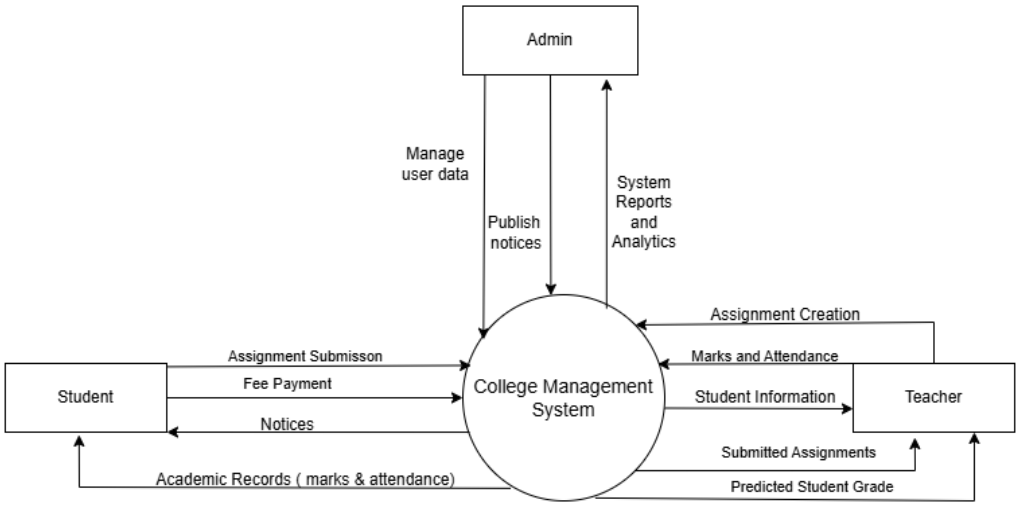


Figure 2 Context Diagram of College Management System

This diagram illustrates the entire system as a single process interacting with external entities, namely the Administrator, Teacher, and Student. It shows the high-level inputs (e.g., grade entry, assignment uploads, fee payments) and outputs (e.g., dashboards, grade predictions, insights) flowing in and out of the centralized system.

DFD Level 1:

This diagram breaks down the main system into detailed sub-processes, mapping exactly how data moves between the User Authentication module, the Academic Record module,

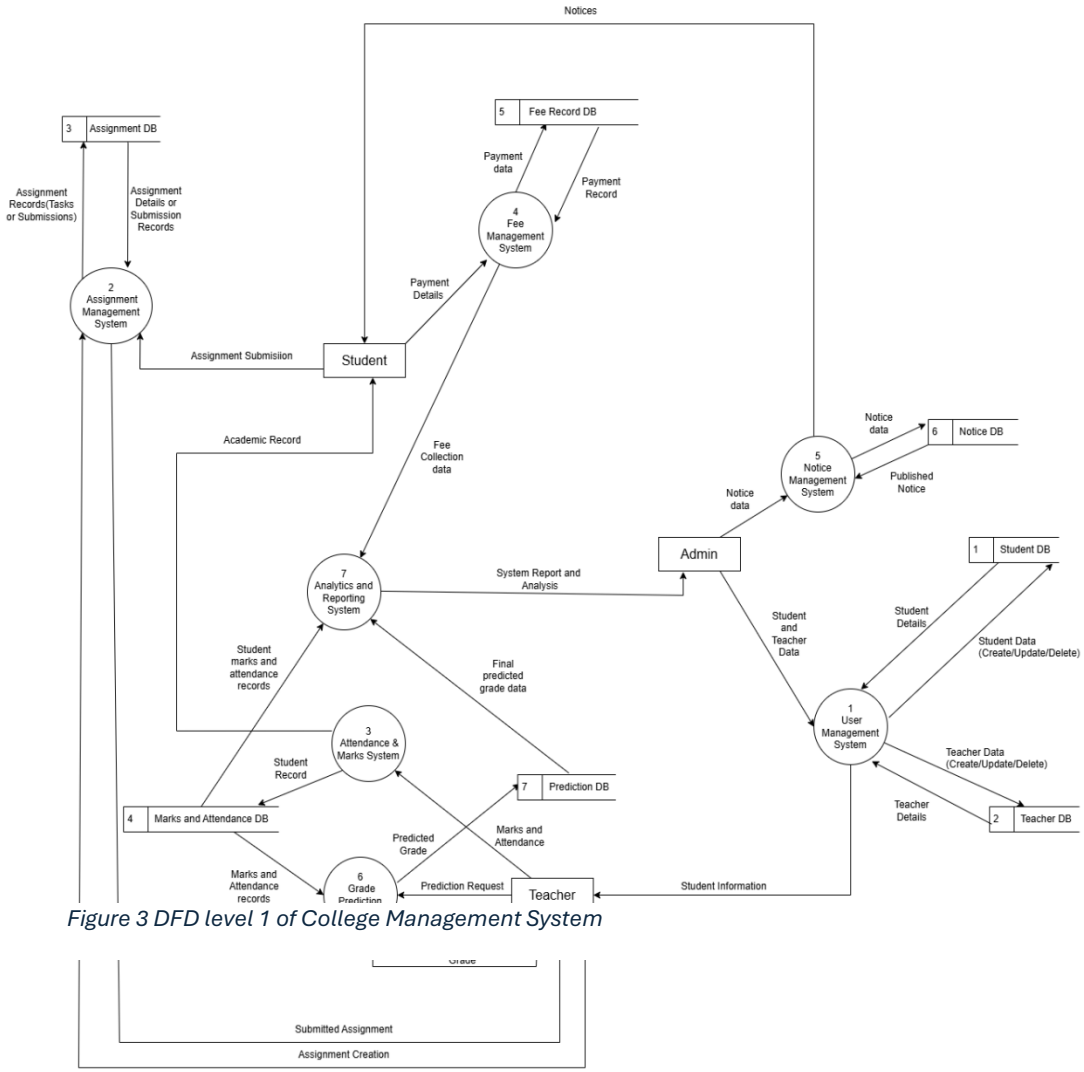


Figure 3 DFD level 1 of College Management System

the Assignment & Fee modules, and the Machine Learning Prediction Engine. It clarifies how data stores (like the student database) interact with these individual processes.

Entity Relationship Diagram

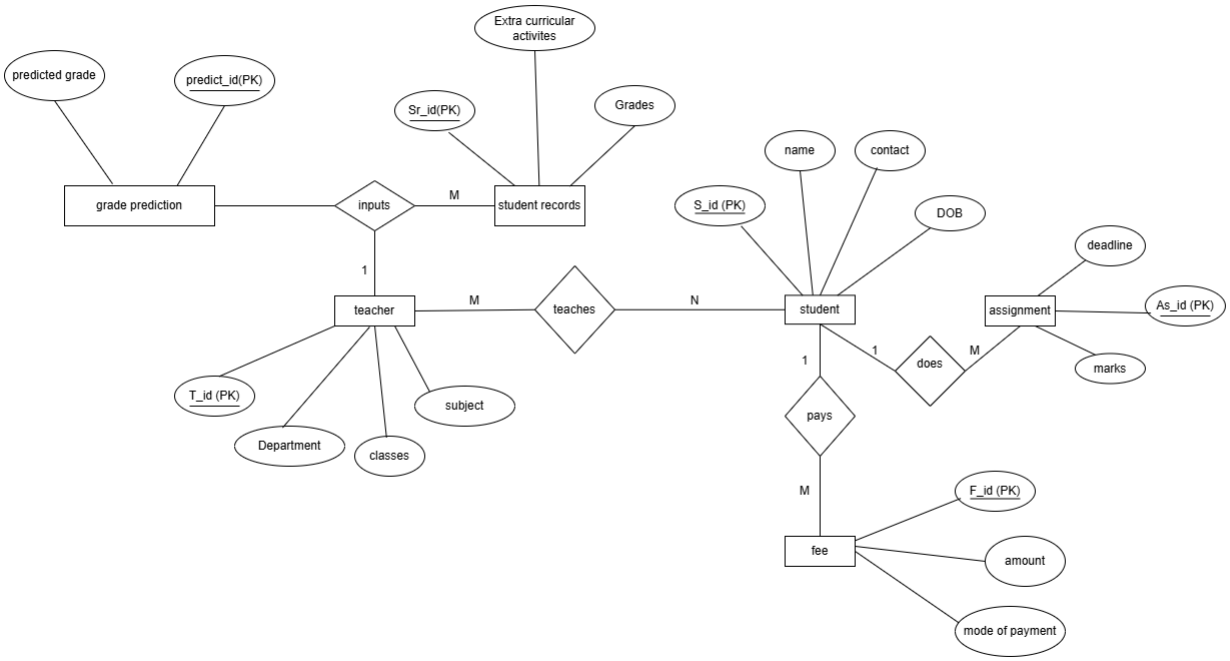


Figure 4 Entity-Relationship Diagram

Data Modelling using ER Diagrams

The Entity-Relationship (ER) diagram provides a logical blueprint of the system's database architecture. It defines the core entities, their specific attributes, and the cardinal relationships connecting them, ensuring that data is stored efficiently without redundancy. Based on the system's structural design, the primary components are mapped as follows:

Core Entities and Attributes:

- Student: Stores personal information with attributes including S_id (Primary Key), name, contact, and DOB (Date of Birth).
- Teacher: Stores faculty details with attributes including T_id (Primary Key), Department, classes, and subject.
- Student Records: Stores academic tracking data with attributes like Sr_id (Primary Key), Grades, and Extra curricular activities.

- Grade Prediction: Stores the machine learning outputs with attributes including predict_id (Primary Key) and the final predicted grade.
- Assignment: Stores task details with attributes including As_id (Primary Key), deadline, and marks.
- Fee: Stores financial transactions with attributes including F_id (Primary Key), amount, and mode of payment.

4 System Design

4.1 Design

This phase translates the requirement specifications and preliminary system analysis into a concrete technical blueprint. Following a structured analysis approach, the design phase focuses on transforming the logical ER diagrams into relational database tables, outlining the user interfaces, and defining system inputs and outputs.

4.1.1 Database Design: Transformation of ER to Relations and Normalizations

The logical Entity-Relationship (ER) models are transformed into relational schemas. To eliminate data redundancy and ensure data integrity, the tables are normalized up to the Third Normal Form (3NF). The database architecture is divided into three core functional modules:

Module for Administration & Setup This module handles the core configuration, user roles, and system-wide announcements.

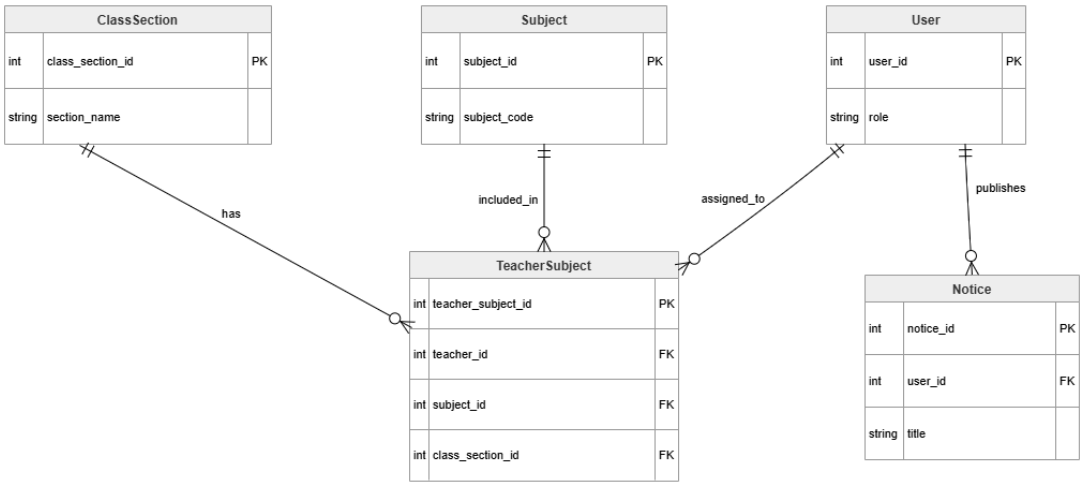


Figure 5 Relational Schema Diagram of administration and setup module

- **User Relation:** Stores authentication and role data. The primary key is user_id, with attributes defining the user's role (Admin, Teacher, or Student).
- **Subject & ClassSection Relations:** The Subject table stores course details (subject_id PK, subject_code), while ClassSection manages grouping (class_section_id PK, section_name).
- **TeacherSubject Relation:** Acts as a junction table to map which teacher instructs which subject in which section. It uses teacher_subject_id as the primary key and holds foreign keys linking to User, Subject, and ClassSection.
- **Notice Relation:** Stores system announcements, containing notice_id (PK), title, and a foreign key (user_id) linking back to the user who published it.

Module for Academic Performance & Daily Tracking This module manages the core academic workflows, including assignments, attendance, and machine learning outputs.

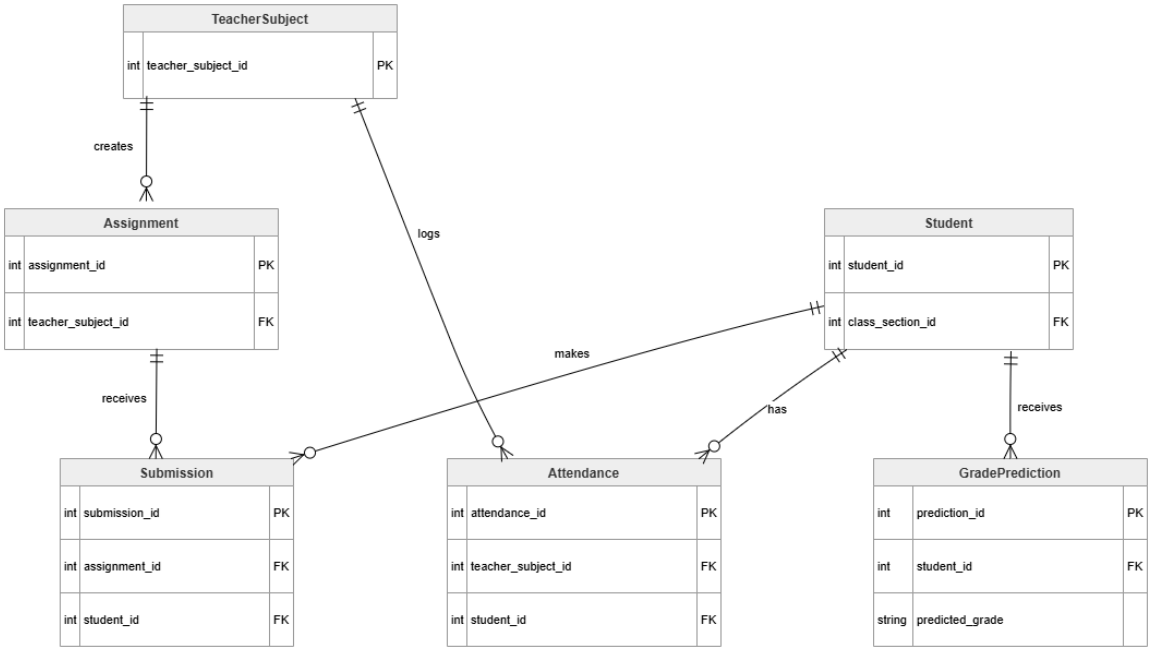


Figure 6 Relational Schema for Academic Performance and Daily Tracking

- **Student Relation:** Stores individual student records, utilizing student_id (PK) and a foreign key to class_section_id.

- **Assignment & Submission Relations:** The Assignment table is linked to the teacher who created it via a foreign key (teacher_subject_id). When a student completes work, it is recorded in the Submission table, linking the assignment_id and student_id via foreign keys.
- **Attendance Relation:** Tracks daily presence, containing an attendance_id (PK) and foreign keys linking the specific student_id to the respective teacher_subject_id.
- **GradePrediction Relation:** Stores the machine learning outputs. It uses prediction_id (PK) and links to the student_id, storing the specific predicted_grade.

•

Module 3: Finance & Billing This module streamlines the financial operations between the college and the students.

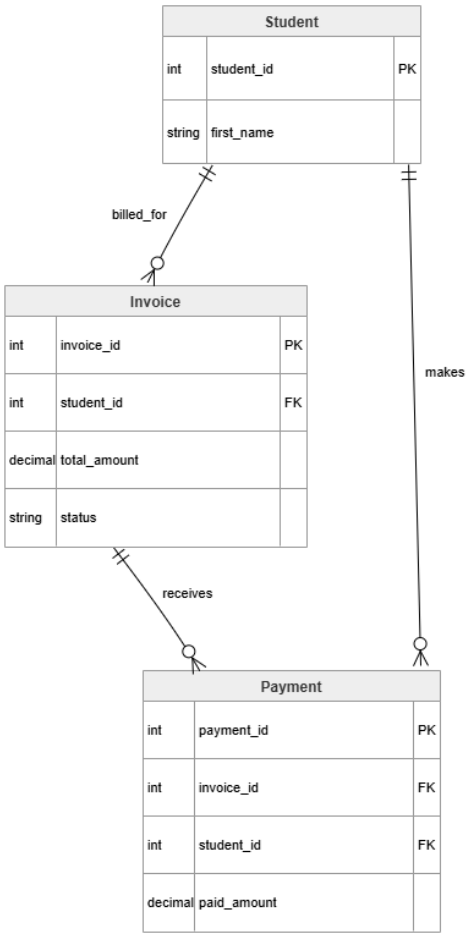
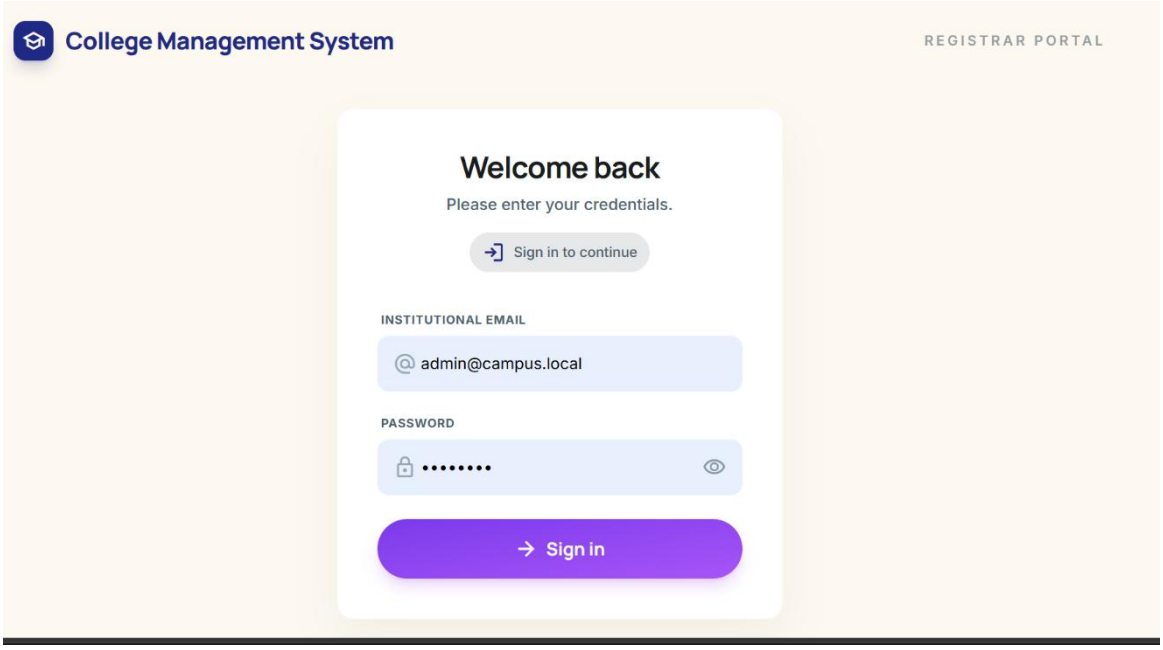
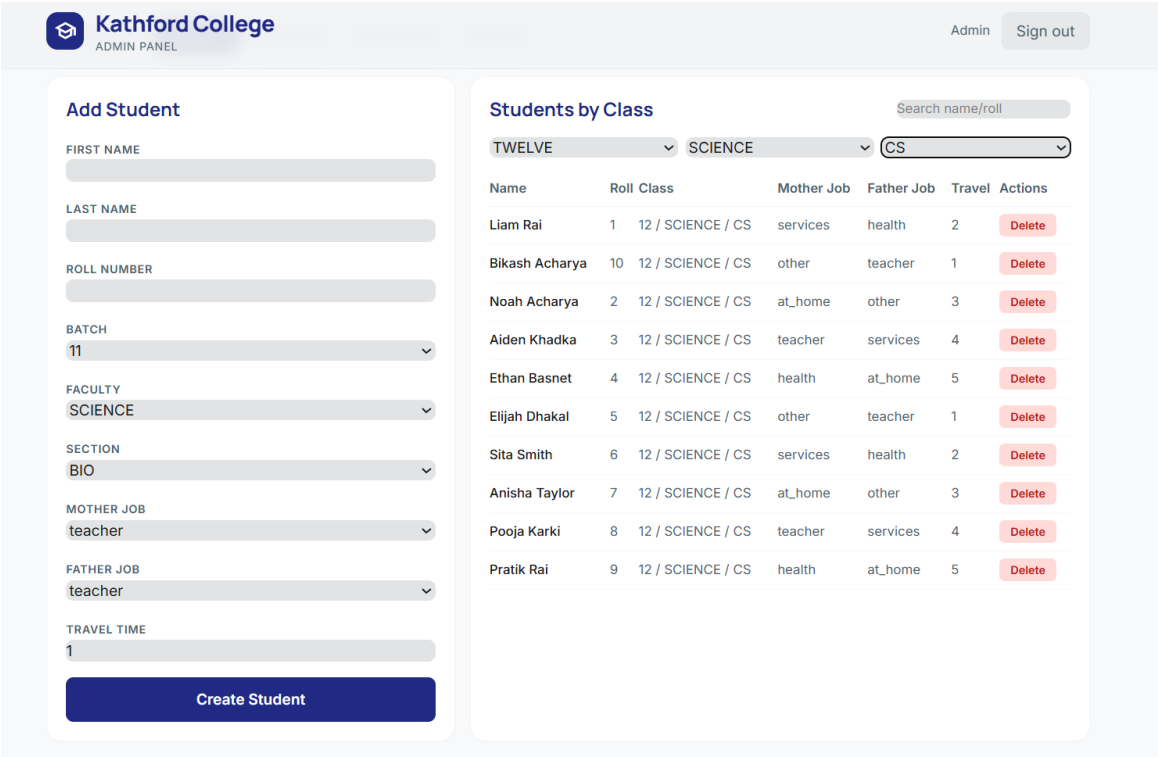


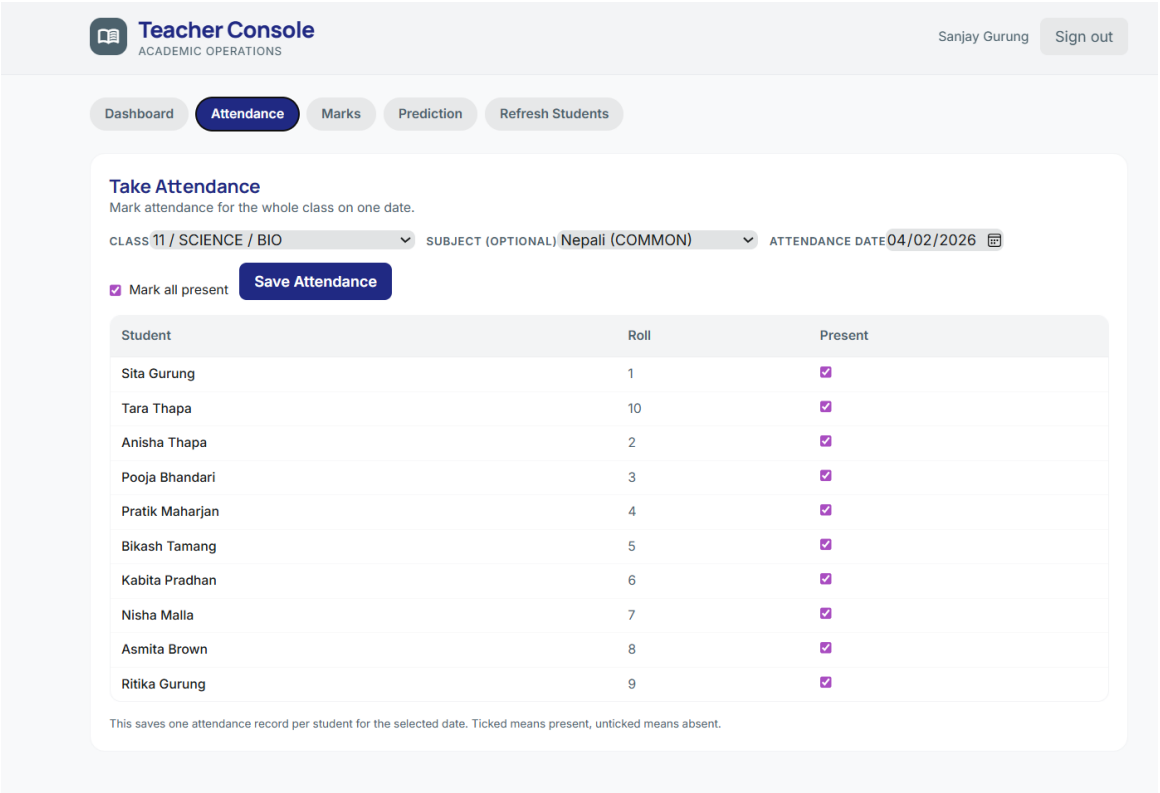
Figure 7 Relational Schema for Fee module

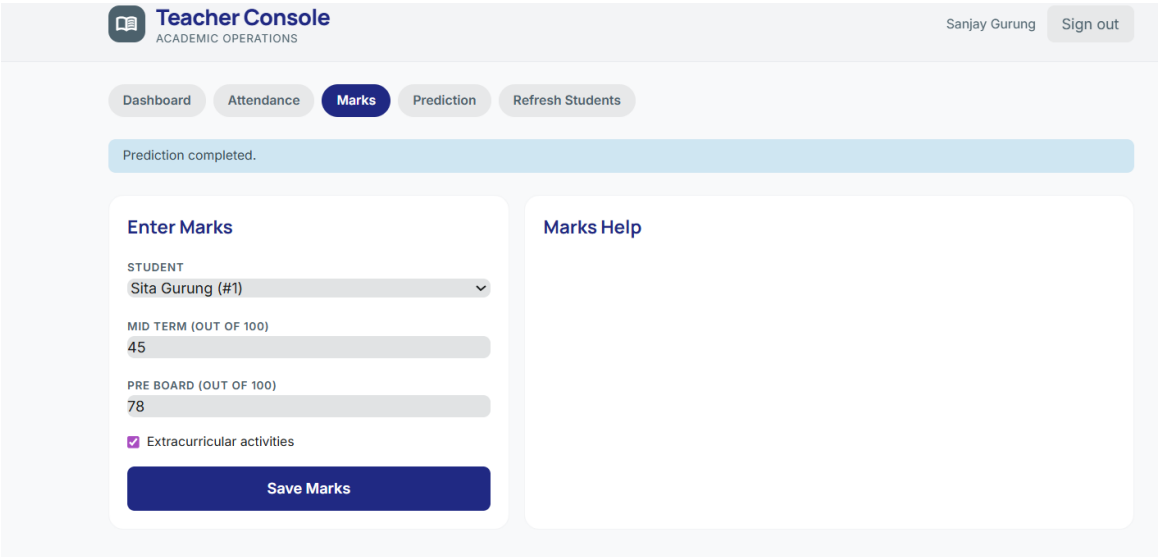
- **Invoice Relation:** Generates the fee bills for students. It contains invoice_id (PK), total_amount, billing status, and a foreign key linking to the respective student_id.
- **Payment Relation:** Tracks transactions made against invoices. It contains payment_id (PK), paid_amount, and foreign keys linking back to both the invoice_id and the student_id.

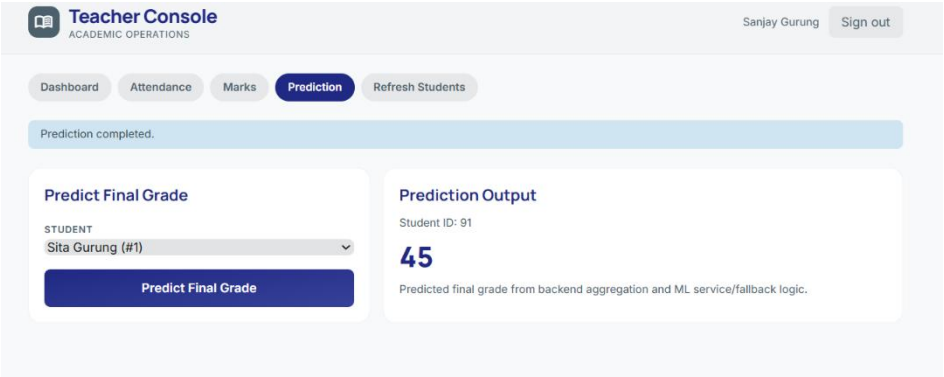
FORM AND REPORT DESIGN






Teacher Console
ACADEMIC OPERATIONS
Sanjay Gurung
Sign out
Dashboard
Attendance
Marks
Prediction
Refresh Students
Take Attendance
Mark attendance for the whole class on one date.
CLASS 11 / SCIENCE / BIO
SUBJECT (OPTIONAL) Nepali (COMMON)
ATTENDANCE DATE 04/02/2026
Mark all present
Save Attendance
Student
Roll
Present
Sita Gurung 1
Tara Thapa 10
Anisha Thapa 2
Pooja Bhandari 3
Pratik Maharjan 4
Bikash Tamang 5
Kabita Pradhan 6
Nisha Malla 7
Asmita Brown 8
Ritika Gurung 9
This saves one attendance record per student for the selected date. Ticked means present, unticked means absent.


Teacher Console
ACADEMIC OPERATIONS
Sanjay Gurung
Sign out
Dashboard
Attendance
Marks
Prediction
Refresh Students
Prediction completed.
Enter Marks
STUDENT
Sita Gurung (#1)
MID TERM (OUT OF 100)
45
PRE BOARD (OUT OF 100)
78
Extracurricular activities
Save Marks
Marks Help


Teacher Console
ACADEMIC OPERATIONS
Sanjay Gurung
Sign out
Dashboard
Attendance
Marks
Prediction
Refresh Students
Prediction completed.
Predict Final Grade
STUDENT
Sita Gurung (#1)
Predict Final Grade
Prediction Output
Student ID: 91
45
Predicted final grade from backend aggregation and ML service/fallback logic.

4.2 Algorithm Details

Random Forest

The predictive engine utilizes the **Random Forest Regressor** to determine a student's final grade ($G3$) based on input features such as attendance, periodic marks ($G1, G2$), and assignment completion rates.

Logic of the Ensemble Method

Random Forest is an "ensemble" learning method, meaning it combines the predictions of many individual decision trees to reach a single, more accurate result. Each tree in the forest is grown by asking a series of binary questions about the student data (e.g., "Is the student's attendance above 80%?"). Since individual trees can sometimes be biased or overfit to specific data points, the Random Forest takes the average of all trees to produce a stable and reliable final grade prediction.

Node Splitting and Error Reduction

During the training phase, the algorithm looks at the Kaggle dataset and decides how to "split" the data into smaller groups. It chooses splits that best separate students with high grades from those with low grades. By repeatedly narrowing down these groups, the algorithm reduces the "error" in its predictions until it reaches a final leaf node that represents the most likely grade for a student with those specific characteristics.

Feature Importance and Performance Analysis

A core benefit of using Random Forest for this College Management System is its ability to perform **Feature Importance** calculation. The algorithm tracks which variables (like attendance, test scores, or assignment submissions) were most effective at reducing prediction error. The system then translates this data into **Performance Factor Analysis**, identifying for each student whether their predicted grade is being primarily influenced by their attendance habits, their exam performance, or their consistency in submitting assignments.

5 Implementation

5.1 Implementaion

5.1.1 Tools Used

The development of the Integrated Academic Information System involved a variety of CASE (Computer-Aided Software Engineering) tools and technologies to ensure structured development and data integrity.

Programming Languages:

- **TypeScript/JavaScript (Node.js):** Chosen for the backend API due to its non-blocking I/O model, which handles multiple concurrent student/teacher requests efficiently.
- **HTML5/CSS3/React:** Used for the frontend to create a responsive user interface for both desktop and mobile views.

Database Platforms:

- **PostgreSQL:** A robust Relational Database (RDBMS) used to implement the 12-table schema. It ensures ACID compliance for student records and attendance logs.

CASE Tools:

- **Visual Studio Code:** The primary Integrated Development Environment (IDE).
- **Postman:** Used as an API testing tool to verify backend endpoints before UI integration.
- **Draw.io:** Used for Architectural and Relational Database modeling.
- **Git:** Utilized for version control to track changes in the codebase.

5.1.2 Implementation Details of Modules

This section describes the core logic and algorithms used to make the system functional.

1. Authentication & Security (BCrypt Algorithm) To protect user data, the system does not store plain-text passwords.

- **Process:** When a user (Admin/Teacher) is created, the system uses the **BCrypt hashing algorithm**.

- **Logic:** It applies a "Salt" to the password and hashes it multiple times. During login, the system compares the hashed input with the stored hash.
- **Method:** verifyPassword(plainText, hashedValue)

2. Academic Mapping Logic (The Junction Algorithm) The most complex part of the implementation is linking Teachers to Subjects and Classes.

- **Procedure:** A central procedure handles the creation of a TeacherSubject record.
- **Logic:** It validates that the teacher_id, subject_id, and class_section_id all exist before creating the link. This prevents "Orphan Records" where a teacher might be assigned to a non-existent class.

3. Attendance Tracking Procedure

- **Method:** recordAttendance(studentList, status, date)
- **Logic:** The system loops through an array of Student IDs for a specific TeacherSubject ID. For each student, it inserts a row into the Attendance table. If a record for that date already exists, the system uses an **UPSERT** (Update or Insert) logic to prevent duplicate entries for the same day.

4. Data Normalization & Validation (Middleware)

- **Algorithm:** Before any data reaches the database, it passes through a **Validation Middleware**.
- **Functions:**
  - emailValidator(): Uses Regular Expressions (Regex) to ensure emails are in the correct format.
  - inputSanitizer(): Strips HTML tags from inputs to prevent SQL Injection and Cross-Site Scripting (XSS) attacks.

5.2 Testing

Testing is a critical phase of the Software Development Life Cycle (SDLC) used to ensure that the "Administration" and "Academic" modules function according to the requirements. A **Bottom-Up testing strategy** was employed, starting with individual code components and ending with full user-level workflows.

5.2.1 Test Cases for Unit Testing

Table 1 Unit Testing

Test ID	Unit/Function	Objective	Test Input	Expected Result	Status
UT-01	verifyPassword()	Validate bcrypt comparison	Correct plain text password	Returns True	Pass
UT-02	verifyPassword()	Block unauthorized access	Incorrect password	Returns False	Pass
UT-03	generateJWT()	Check token payload	userId: 10, role: Teacher	Token contains valid ID/Role	Pass
UT-04	authMiddleware()	Test missing credentials	Request without Token	HTTP 401 Unauthorized	Pass
UT-05	calcAttendance()	Handle new students	present: 0, total: 0	Returns 0 (No Error)	Pass
UT-06	validateInput()	Check for empty fields	Name: "", Email: "test@"	Validation Error triggered	Pass

Unit testing focuses on the smallest parts of the application, such as individual functions or database queries, to ensure they handle data correctly in isolation.

5.2.2. Test Cases for System Testing

System testing validates the complete integrated software. It ensures that different modules (like Users and Class Sections) work together to complete a business process

Table 2 System Testing

Test ID	Module/Scenario	Preconditions	Execution Steps	Expected Result	Status
ST-01	Admin Login	Account exists	Enter valid Email/Pass	Redirect to Admin Home	Pass
ST-02	Teacher Setup	Admin is logged in	Create Teacher & Assign Subject	Teacher/Subject linked	Pass
ST-03	Student Enrollment	Class Section exists	Add Student to Grade 10-A	Student record created	Pass
ST-04	Attendance Entry	Teacher is logged in	Mark "Present" for Student A	Attendance record stored	Pass
ST-05	Role Security	Student is logged in	Attempt to access /users/delete	Access Denied (403)	Pass
ST-06	Data Retrieval	Attendance exists	View Attendance Report	Data displays correctly	Pass

5.3 Result Analysis

The results of the testing phase indicate that the core modules of the **Integrated Academic Information System** are functionally sound and ready for deployment in a demonstration environment.

1. Reliability of Core Logic:

The system successfully passed all unit tests related to security and data calculation. Specifically, the password hashing and JWT (JSON Web Token) generation processes ensure that the system's "Front Gate" is secure.

2. Relational Integrity:

System testing confirmed that the "Many-to-Many" relationship between Teachers and Subjects is handled correctly via the TeacherSubject junction table. Data entered in the Admin module successfully flows into the Academic module without loss or corruption.

3. User Experience and Security:

Role-Based Access Control (RBAC) was validated. The system effectively restricts students from accessing teacher functions and teachers from accessing administrative settings, maintaining strict data privacy.

4. Conclusion:

While the AI Prediction and Financial modules are earmarked for future integration testing, the current stable version provides a robust foundation for school administration, attendance tracking, and user management. The software meets all primary objectives defined in the project scope.

6 References

[1] S. Agyemang, E. O. Appiah, and E. A. Yirenkyi, "Predicting Students' Academic Performance Via Machine Learning Techniques," ScholarWorks@UTRGV, 2023.

[2] M. Shoaib, N. Sayed, J. Singh, J. Shafi, S. Khan, and F. Ali, "AI student success predictor: Enhancing personalized learning in campus management systems," Computers in Human Behavior, vol. 158, p. 108301, Sep. 2024.

[3] T. Gray, C. Perkins, and R. Lewis, "Using learning analytics to predict student retention in higher education," IEEE Transactions on Learning Technologies, vol. 14, pp. 115-128, 2021.

[4] J. Xu and H. Yang, "A comparative study of Random Forest and Multiple Linear Regression models in predicting student academic performance," Expert Systems with Applications, vol. 185, p. 115629, 2022.

[5] W. A. Sajid, N. Mirzah, F. M. Mustafa, and Y. Shabala, "Educational Institution Management Information System," in 2024 35th Conference of Open Innovations Association (FRUCT), IEEE, 2024, pp. 625-632.

[6] Paathshala, "School-College Management System Features," 2025. [Online]. Available: https://paathshala.com.np/